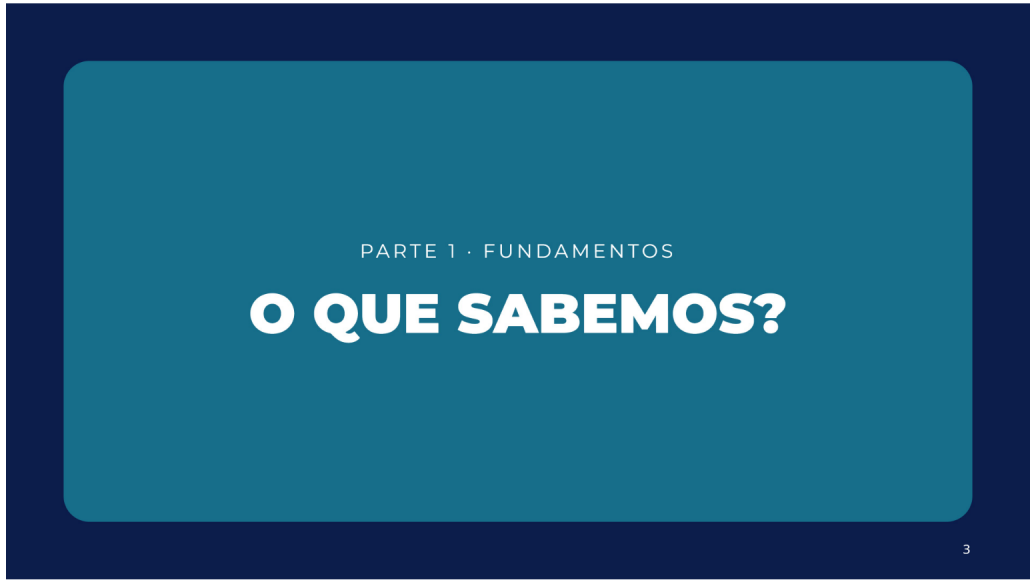
CLOUD COM AWS PARA FLUTTER

Guilherme Rodrigues

Abertura (2 min). Promessa da aula: ao final das 4 horas, cada aluno terá um app Flutter lendo e gravando dados numa API que ele mesmo criou na AWS. As 2 horas de teoria são divididas em duas partes: fundamentos de cloud e serviços da AWS.



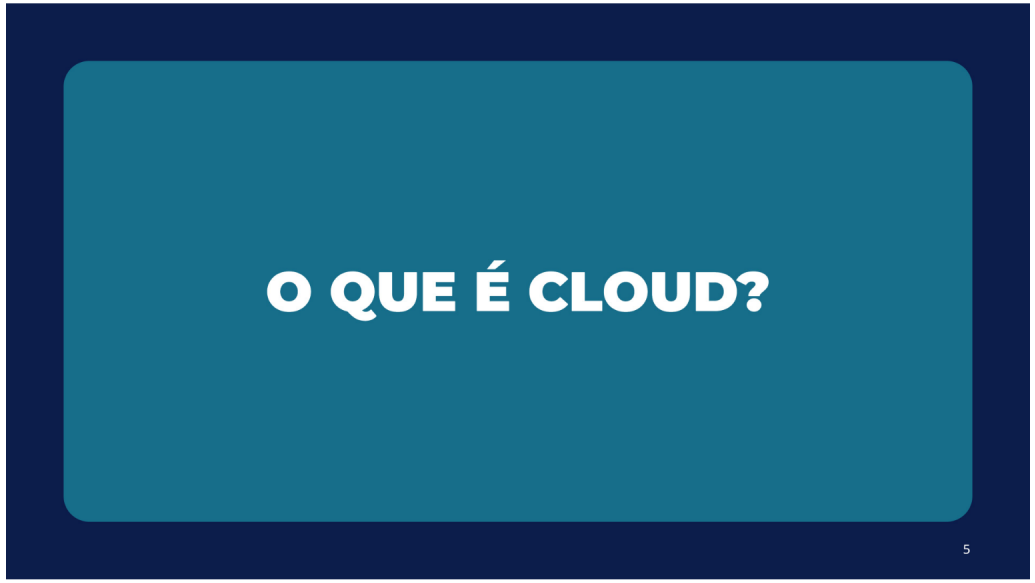
Mapa da aula (1 min). Parte 1 (0:05–0:55) é para quem nunca teve contato com nuvem: nada de código, só conceitos e exemplos do dia a dia. Intervalo de 10 min. Parte 2 (1:05–1:55) apresenta sete serviços da AWS sempre pela mesma pergunta: o que é, para que serve, onde entra num app Flutter. Detalhes de HTTP, async e CORS ficaram no apêndice, para o começo da prática.



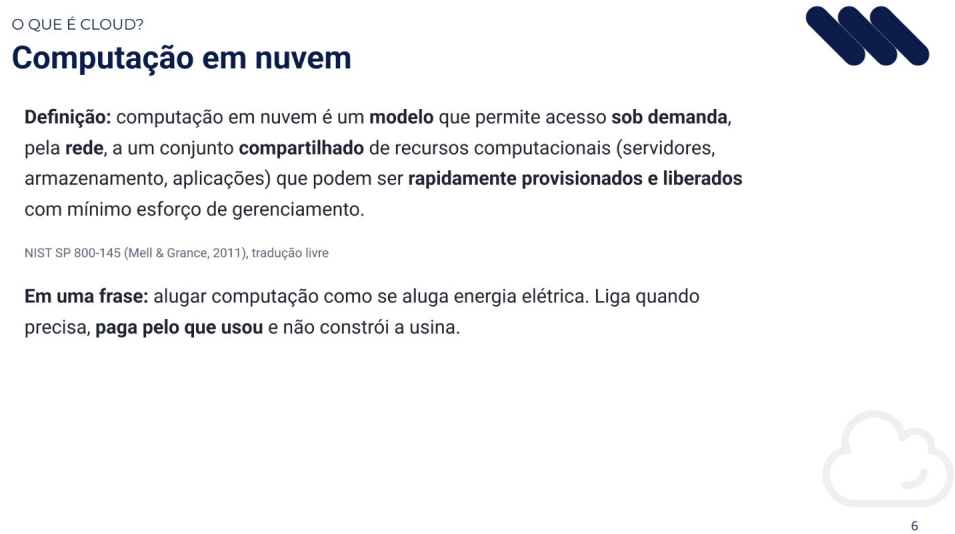
Início da Parte 1 (50 min). Antes de qualquer definição, pergunte o que a turma entende por "nuvem". Anote as palavras no quadro (Drive, servidor, internet, armazenamento); elas voltam no slide de definição. Público-alvo: quem nunca ouviu falar de nuvem além do "salvar no Drive".



Para que serve (5 min). Comece pelo que eles já usam, antes de qualquer definição. Pergunte: "se você trocar de celular hoje, o que você perde?" Quase nada, porque fotos, conversas, e-mails e saldo estão em servidores. O app no celular é só a vitrine. Essa frase ("o app é a vitrine, a nuvem é o estoque") volta na Parte 2.



Transição para a definição formal. Retome as palavras anotadas no quadro.



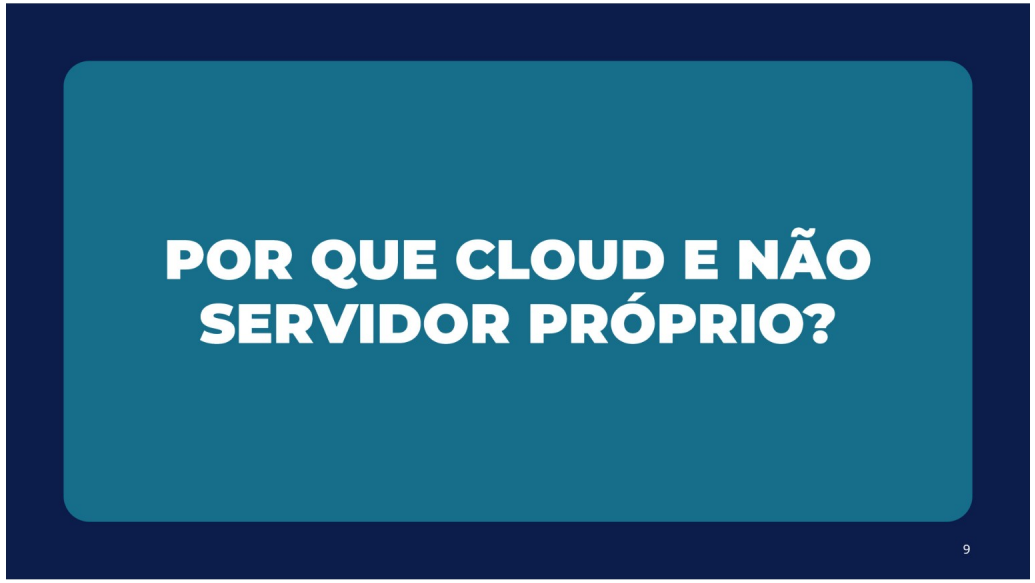
Definição (4 min). Use a definição do NIST porque é a referência citada em praticamente todo livro e prova de concurso. Destaque as palavras em negrito: "sob demanda", "compartilhado" e "rapidamente provisionados". A "nuvem" continua sendo computador de alguém, em um data center real; o que muda é o modelo de acesso e de cobrança. A analogia da energia elétrica é útil, mas aponte o limite dela: diferente da tomada, na nuvem você precisa configurar, proteger e desligar o que liga.



CAPEX vs OPEX (3 min). CAPEX = despesa de capital (compra de ativo, que entra no patrimônio e é depreciado ao longo dos anos); OPEX = despesa operacional (conta recorrente). Exemplo para fixar: comprar um carro é CAPEX; pegar Uber é OPEX. Seja honesto sobre a contrapartida: cloud não é automaticamente mais barata. Para carga estável e previsível, servidor próprio pode sair mais em conta; a vantagem da nuvem é a velocidade e a elasticidade. O último item do lado direito prepara o slide de custos.



As 5 características + escalabilidade (6 min). Faça a turma testar o critério: "um servidor alugado por mês numa hospedagem tradicional é cloud?" Geralmente falha em elasticidade e em serviço medido. "O Google Drive é cloud?" Sim, como SaaS. Nomes originais do NIST: on-demand self-service, broad network access, resource pooling, rapid elasticity, measured service. Diferença que costuma confundir: escalabilidade é a CAPACIDADE de crescer (pode ser planejada e manual, pensando no longo prazo: "meu sistema aguenta 10x mais usuários se eu adicionar máquinas?"); elasticidade é crescer E encolher SOZINHO, rápido, conforme a demanda do momento (curto prazo). Todo sistema elástico é escalável, mas nem todo sistema escalável é elástico: um servidor próprio pode ser escalável (você compra mais máquinas), mas não encolhe depois da Black Friday. Escalar horizontal = mais máquinas iguais (scale out); vertical = máquina maior (scale up). Ligue "serviço medido" ao slide de custos.



Pergunta para a turma antes de mostrar a tabela: "se você fosse lançar um app semana que vem, compraria servidores?"

POR QUE CLOUD?

Servidor próprio vs. cloud

Critério	Servidor próprio	Cloud
Para começar	Comprar máquinas, sala, energia	Nada: paga conforme usa
Servidor novo	Semanas	Minutos
Pico de acesso	Cai, ou sobra máquina o ano todo	Cresce no pico e encolhe depois
Manutenção física	Sua equipe: peças, energia, ar	Do provedor
Usuários no mundo	Um endereço só	Regiões em vários continentes
Controle do hardware	**Total**	Nenhum
Uso grande e constante	**Pode sair mais barato**	Pode sair mais caro

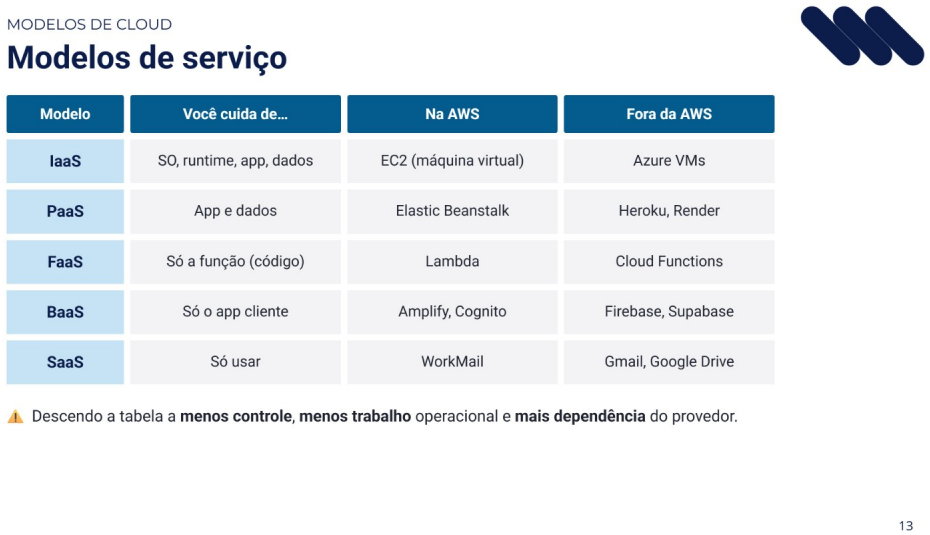
Por que cloud (6 min). Faça a turma pensar como dona de uma startup que vai lançar um app semana que vem: quanto precisa gastar antes do primeiro usuário? Com servidor próprio, muito; na nuvem, quase nada. O argumento mais forte não é preço, é tempo e risco: dá para testar uma ideia barato e, se der certo, crescer sem trocar de casa. Mas termine nas duas linhas teal: a nuvem não vence em tudo, o que prepara o próximo slide.



Três classificações: o que você aluga (serviço), de quem é a infraestrutura (implantação) e quem protege o quê (responsabilidade).



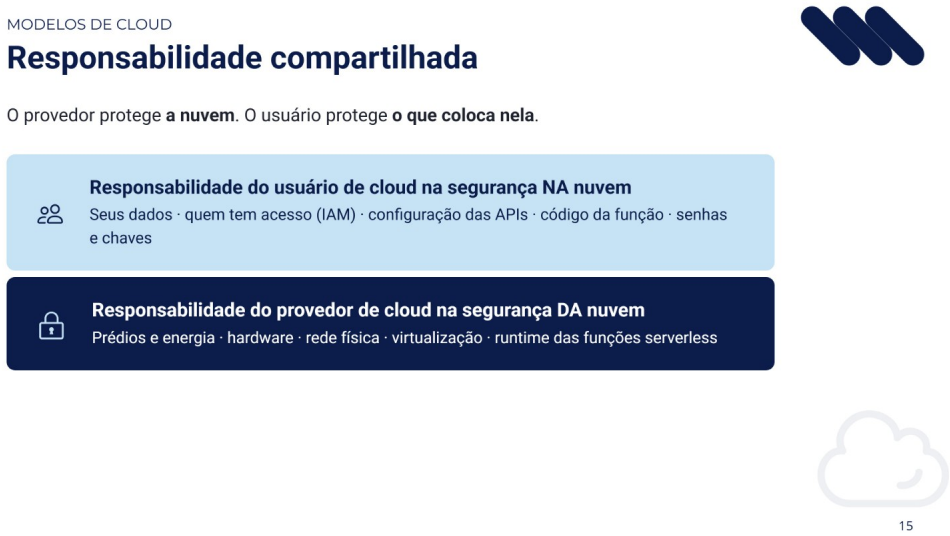
Definições dos modelos de serviço (4 min). Leia de cima para baixo: a cada linha, o provedor assume mais uma camada e você cuida de menos coisas. IaaS, PaaS e SaaS são os três modelos do NIST (SP 800-145); FaaS e BaaS são termos de mercado posteriores, mas são justamente os que interessam a quem faz apps. O slide seguinte mostra exemplos de cada um na AWS e fora dela.



Modelos de serviço (6 min). A analogia da pizza ajuda: IaaS = você aluga a cozinha; PaaS = compra a massa pronta e assa; FaaS = só escolhe o sabor de cada fatia quando alguém pede; SaaS = pede pronta. Os livros clássicos falam só de IaaS/PaaS/SaaS; FaaS e BaaS são termos posteriores, mas são justamente os que interessam a quem faz apps. Diga explicitamente onde a prática se encaixa: FaaS (Lambda) com um banco gerenciado (DynamoDB). Firebase, que eles já viram, é BaaS.



Modelos de implantação (3 min). São os quatro modelos do NIST. Mencione "multicloud" (usar mais de um provedor público) como termo de mercado, não do NIST. Pergunta rápida: o app de um banco brasileiro provavelmente usa qual modelo, e por quê? A resposta costuma envolver regulação e dados sensíveis (LGPD, Banco Central).



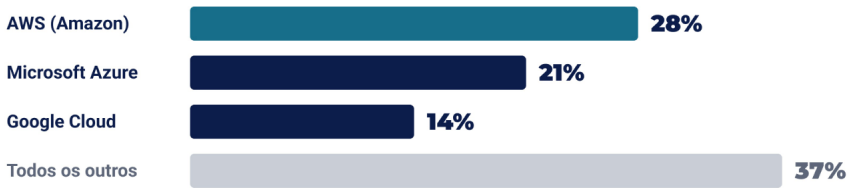
Responsabilidade compartilhada (4 min). É o conceito mais cobrado em certificações da AWS. A linha entre as camadas sobe conforme o modelo: no EC2 o sistema operacional é seu (você aplica patches); na Lambda o runtime é da AWS. Exemplo concreto: se alguém deixa uma API sem autenticação e um terceiro apaga os dados, a culpa não é da AWS. A API da prática NÃO tem autenticação, de propósito, para caber em 2h; retome isso no slide do Cognito.



Antes de mostrar: peça para a turma chutar quais apps do celular deles rodam em nuvem.

QUEM USA CLOUD?

Infraestrutura na nuvem



Participação no gasto mundial com infraestrutura de nuvem no 1º trimestre de 2026: **US$ 129 bilhões** no trimestre, 35% a mais que um ano antes. Fonte: Synergy Research Group, via CRN.

Provedores (4 min). AWS lidera desde que criou o mercado (2006), mas vem perdendo participação aos poucos, enquanto o Google Cloud cresce. Os conceitos da aula valem para os três: EC2 tem equivalente no Azure (Virtual Machines) e no Google (Compute Engine); Lambda tem Azure Functions e Cloud Functions; e assim por diante. Escolhemos AWS por ser a maior e a mais pedida em vagas, não por ser a única opção. Os números são de um trimestre e mudam a cada relatório; atualize antes de cada oferta.



Quem usa (6 min). Três padrões diferentes: Netflix migrou (saiu de servidor próprio para a nuvem); Nubank nasceu na nuvem; Spotify mostra que AWS não é a única opção. Nuance sobre a Netflix: o vídeo em si chega por uma rede de entrega própria (Open Connect), com caixas instaladas dentro dos provedores de internet; na AWS ficam o catálogo, contas, recomendações e análise de dados. Até a empresa símbolo da nuvem é híbrida no que mais importa para ela. Fontes nos estudos de caso da AWS (Netflix, Nubank, iFood), no ComputerWeekly (Netflix 2016) e no blog do Google Cloud (Spotify 2016); estão em Referências.



Início da Parte 2 (50 min). Combine o formato com a turma: cada serviço terá um slide com o mesmo desenho (o que é, para que serve, no app Flutter, analogia, como cobra). Não é para decorar nomes; é para saber qual peça procurar quando o app precisar de login, de fotos ou de um banco.



Amazon EC2 (4 min). EC2 = Elastic Compute Cloud, um dos serviços mais antigos da AWS. Cada máquina é uma "instância"; o "tipo de instância" define CPU e memória (ex.: t3.micro, pequena e barata, usada em contas de estudo). Ponto-chave: a AWS cuida do hardware, você cuida de tudo do sistema operacional para cima (atualizações, segurança, instalação). Alerta de custo: a cobrança corre enquanto a máquina estiver ligada, mesmo sem nenhum acesso. Os equivalentes têm o mesmo papel: Azure Virtual Machines (Microsoft) e Compute Engine (Google Cloud). Quem aprende um entende os outros.



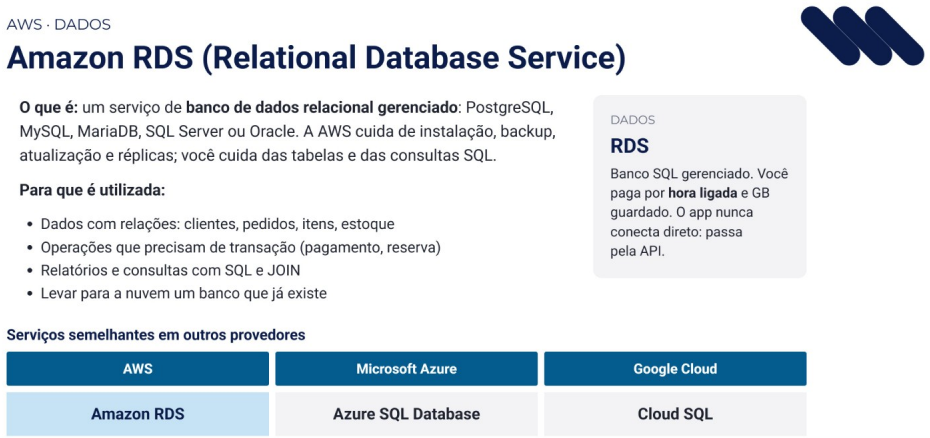
AWS Lambda (4 min). "Serverless" não significa sem servidor: significa que o servidor não é problema seu. Limites para mencionar sem aprofundar: a função não guarda nada entre uma chamada e outra (por isso precisa de banco), a primeira chamada depois de um tempo parada demora um pouco mais (cold start) e há teto de 15 minutos por execução. Lambda não roda Dart nativamente; na prática usaremos Python, com o código pronto. Equivalentes: Azure Functions (Microsoft) e Cloud Run functions no Google Cloud, que até 2024 se chamava Cloud Functions; muitos materiais ainda usam o nome antigo, e o slide de modelos de serviço também. Confira o nome atual antes da aula.



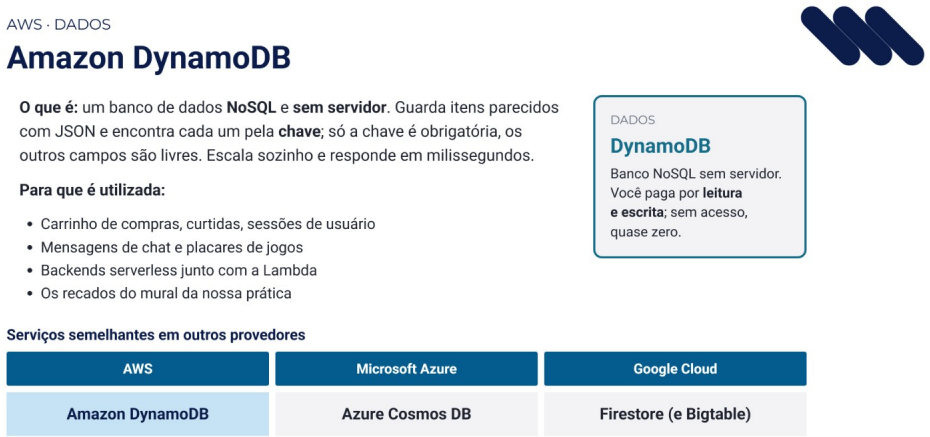
Amazon API Gateway (4 min). Sem ele, a Lambda não tem endereço na internet. Na prática usaremos o tipo "HTTP API", mais simples e barato que o "REST API" antigo. Alternativa que aparece em tutoriais: Lambda Function URL, uma URL direto na função, mais simples, mas sem rotas nem controle de acesso. Equivalentes: Azure API Management (Microsoft); no Google Cloud há o API Gateway, mais simples e voltado a funções serverless, e o Apigee, plataforma corporativa de gestão de APIs. CORS está detalhado no apêndice.



Amazon S3 (4 min). Regra de ouro: o banco de dados guarda o endereço da foto, o S3 guarda a foto. Ninguém salva imagem dentro do DynamoDB ou do RDS. No app Flutter, o jeito seguro de enviar um arquivo é a URL assinada (presigned URL): o backend gera um link que vale por poucos minutos e o app faz o upload direto nele, sem nunca carregar credenciais da AWS. Vazamentos famosos de dados vieram de buckets deixados públicos por engano; por padrão a AWS bloqueia acesso público. Equivalentes: Azure Blob Storage (Microsoft) e Cloud Storage (Google Cloud), que muitos usam com Flutter via Firebase Storage.



Amazon RDS (4 min). Para quem já viu banco de dados na graduação, é o serviço mais familiar: é o mesmo PostgreSQL ou MySQL, só que alguém cuida dele. Existe também o Aurora, versão da própria AWS compatível com MySQL e PostgreSQL. Segurança: o banco nunca fica exposto ao app ou à internet; quem fala com ele é o backend (Lambda ou EC2). Custo: como o EC2, o RDS cobra enquanto está ligado; é um dos itens que mais estouram conta de estudante. Equivalentes: Azure SQL Database (SQL Server; para PostgreSQL e MySQL há o Azure Database for PostgreSQL/MySQL) e Cloud SQL no Google Cloud (PostgreSQL, MySQL e SQL Server).



Amazon DynamoDB (4 min). Mostre um item do mural: {"id": "a1b2…", "autor": "Ana", "texto": "Olá", "curtidas": 3}. Não existe CREATE TABLE com colunas: a tabela só declara a chave. Limite honesto: buscar pela chave é rapidíssimo; procurar por outro campo ("todos os recados da Ana") fica lento e caro, porque o banco precisa ler tudo. Por isso o desenho das chaves vem antes de tudo. Equivalentes: Azure Cosmos DB (Microsoft); no Google Cloud, Firestore (o banco que muitos já usam com Flutter via Firebase) e Bigtable, este para volumes muito grandes.



Mapa da Parte 2 (3 min). Apresente o mapa antes dos detalhes, agrupando pelo papel e não pelo nome: computação (onde o código roda), entrada (por onde o app chega), arquivos, dados, usuários. Exercício rápido: "num app tipo Instagram, qual serviço guarda a foto? Qual guarda as curtidas? Qual faz o login?" (S3, DynamoDB ou RDS, Cognito). Os slides seguintes seguem o caminho de uma requisição.



Arquivos e bancos: S3, RDS e DynamoDB. Pergunta para abrir: "onde fica a foto de perfil e onde fica o nome do usuário?" São lugares diferentes.

COMPARANDO

EC2 ou Lambda?

EC2 (servidor)	Lambda (serverless)	Fica ligado
24 h por dia	Só durante cada chamada	**Paga por**
Hora ligada	Chamada + milissegundos	**Aguenta pico**
Se você configurar	Sozinha, e volta a zero	**Atualiza o sistema**
Você	AWS	**Bom para**
Tráfego alto e constante, processos longos	Tráfego irregular, tarefas curtas	

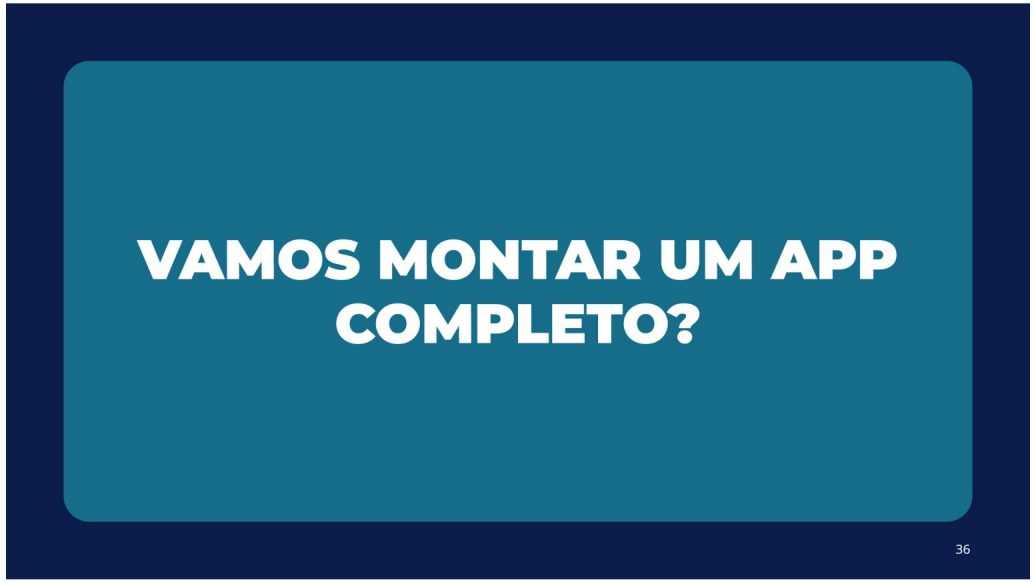
EC2 vs Lambda (3 min). Volte ao contraponto da Parte 1: assim como servidor próprio pode vencer a nuvem com uso grande e estável, EC2 pode vencer Lambda nas mesmas condições. "Serverless" não significa que não há servidor; significa que ele não é problema seu. Pergunta: um app de chat com milhões de mensagens o dia todo ficaria melhor em qual? (Discussão aberta; ambos são defensáveis.)

COMPARANDO

RDS ou DynamoDB?

RDS (SQL)	DynamoDB (NoSQL)	Formato
Tabelas com colunas fixas	Itens livres, só a chave fixa	**Consulta**
SQL, JOIN, filtros à vontade	Pela chave; o resto é caro	**Funciona como**
Servidor de banco ligado	Sem servidor	**Paga por**
Hora ligada	Leitura e escrita	**Exemplo**
Loja: clientes, pedidos, estoque	Mural: recados por id	

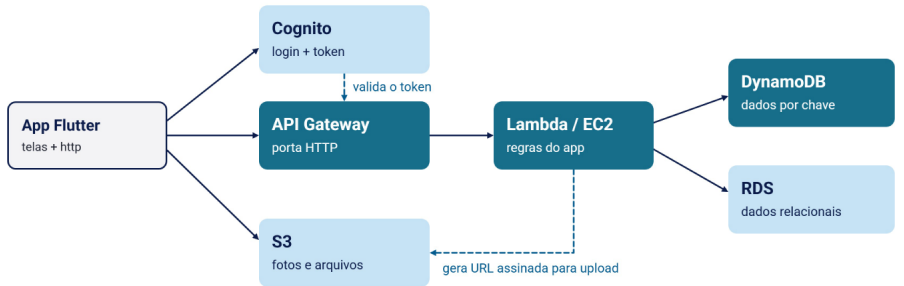
RDS vs DynamoDB (3 min). Não há vencedor universal. A escolha da prática (DynamoDB) foi por ser sem servidor e barato parado, não por ser "melhor". Um e-commerce provavelmente usa RDS para pedidos (precisam de transações e relações) e DynamoDB para o carrinho (muito acesso, estrutura simples).



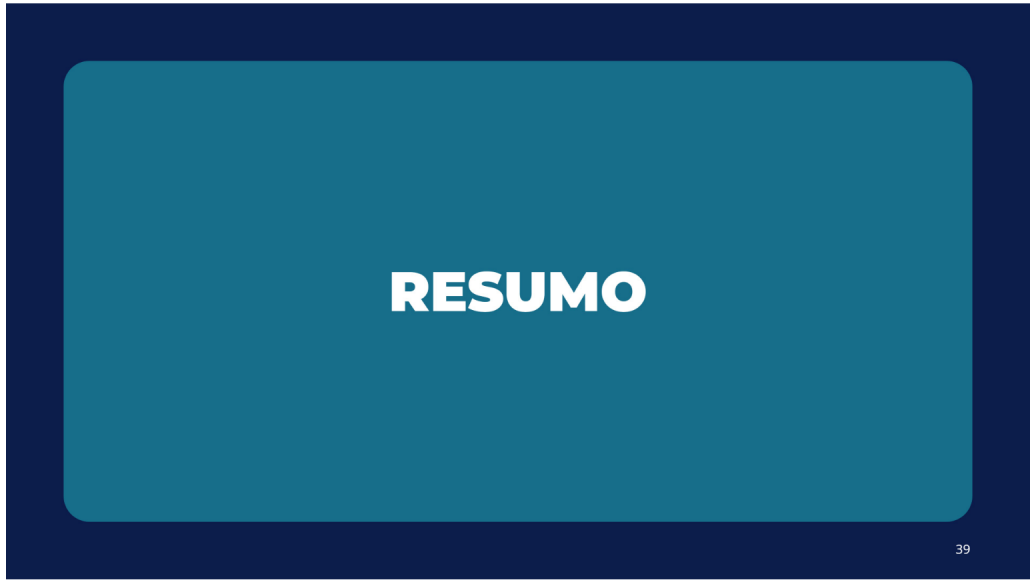
Antes de mostrar o diagrama, desafie a turma: "um app de fotos com login. Quais serviços vocês usariam e como eles se ligam?" Dê 2 minutos para discutirem em duplas.

VAMOS MONTAR UM APP COMPLETO?

Um app Flutter completo na AWS



Juntando tudo (5 min). Percorra dois cenários com o dedo no desenho. (1) Postar uma foto com legenda: o app faz login no Cognito e recebe o token; pede à API uma URL assinada; a Lambda gera a URL; o app envia a foto direto para o S3; depois envia a legenda e o endereço da foto para a API, que grava no DynamoDB. (2) Ver o histórico de pedidos: app → API Gateway (com token) → Lambda → RDS → JSON de volta. Repare que o app só conversa com três coisas: Cognito, API Gateway e S3 (por URL assinada). Banco nunca. Na prática faremos só o caminho em teal.



Fechamento da teoria (5 min).

SERVIÇOS AWS: RESUMO

Serviço	Categoria	Palavra-chave	No app Flutter
EC2	Computação	servidor ligado	API tradicional chamada por HTTP
Lambda	Computação	função por evento	Lógica do backend (prática)
API Gateway	Entrada	URL + rotas	Endereço que o app chama (prática)
S3	Arquivos	objetos em buckets	Fotos enviadas por URL assinada
RDS	Dados	SQL gerenciado	Pedidos e relações, via API
DynamoDB	Dados	chave → item	Recados do mural, via API (prática)
Cognito	Usuários	login + token	Telas de login; token em cada pedido

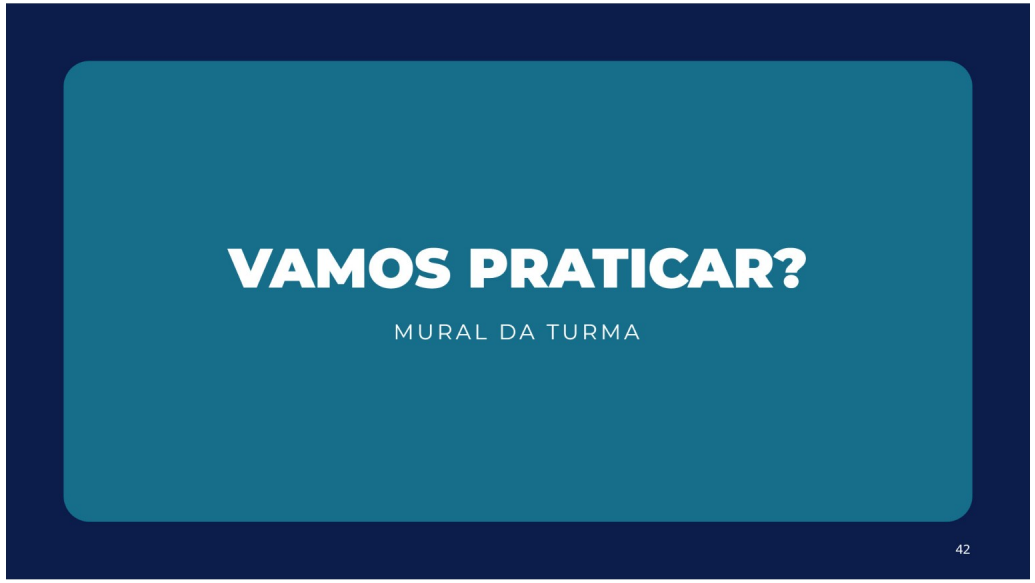
Resumo dos serviços (3 min). Mesmo formato da tabela de resumo das outras aulas. Peça para a turma cobrir a coluna "No app Flutter" e dizer, para cada serviço, onde ele entraria num app que já usam.

OBRIGADO

PELA ATENÇÃO



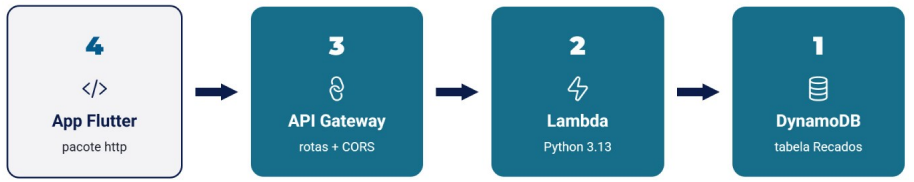
Encerramento da teoria. Se a prática começar em seguida, emende com o apêndice (HTTP, async e CORS).



Ponte para a prática (3 min nos próximos três slides). Pode ser apresentada no fim da teoria ou no começo das 2 horas práticas.

VAMOS PRATICAR? · ARQUITETURA

Quatro peças, montadas da direita para a esquerda



Montamos do banco para o app: cada peça é **testada** antes que a próxima dependa dela.

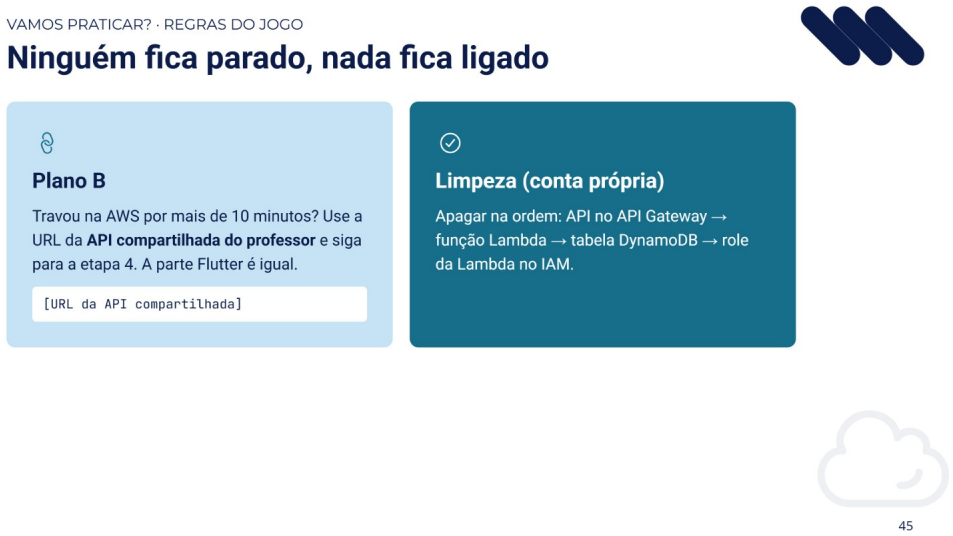
Arquitetura da prática (3 min). Explique por que a ordem é de trás para frente: se o app falhar, já sabemos que banco, função e API funcionam (cada um foi testado), e o problema está no app. É uma lição de depuração que vale para a vida: testar camada por camada. Cada etapa do roteiro termina com um checkpoint.

VAMOS PRATICAR? · AS 2 HORAS

Cada etapa termina com um checkpoint

Minutos	Etapa	Checkpoint
0–10	0 · Preparar ambiente	Console na região certa; app roda vazio
10–20	1 · Tabela DynamoDB	Tabela Recados ativa
20–45	2 · Função Lambda	Teste no console devolve 200
45–65	3 · API Gateway + CORS	URL/recados mostra [] no navegador
65–100	4 · App Flutter	Recado criado aparece para o colega
100–110	Desafio: curtidas	Contador de curtidas sobe
110–120	Limpeza	Recursos apagados

Cronograma da prática (2 min). Ao fim de cada etapa, peça que levantem a mão quem passou do checkpoint; se menos da metade passou, resolva no telão antes de avançar. A etapa 4 tem folga maior porque é onde aparecem os erros de CORS e de digitação em Dart. O checkpoint da etapa 4 é social de propósito: ver o recado do colega é a prova de que o estado está na nuvem. Se usar o apêndice (HTTP, async, CORS) no começo da prática, desconte 10 minutos da etapa 4 ou do desafio.



Plano B e limpeza (2 min). Suba a API compartilhada antes da aula com o template CloudFormation da pasta backend (template-plano-b.yaml) e escreva a URL neste slide. Combine o critério: mais de 10 minutos travado na AWS = plano B, sem culpa. Na API compartilhada todos podem apagar recados de todos, porque não há autenticação; transforme o vandalismo inevitável em lição sobre responsabilidade compartilhada. No Learner Lab, a limpeza é menos crítica (o orçamento é da turma), mas o hábito importa.

REFERÊNCIAS

Para estudar depois

Conceitos e mercado

- AWS: estudo de caso Nubank
- AWS: estudo de caso iFood
- Google Cloud: Spotify escolhe o GCP

Serviços AWS e Flutter

- Amazon EC2 · AWS Lambda
- API Gateway: HTTP APIs
- Amazon S3 · Amazon RDS
- Amazon DynamoDB
- Amazon Cognito
- Responsabilidade compartilhada
- Flutter: buscar dados da internet
- AWS Free Tier · AWS Academy

Referências. Números de mercado, clientes e preços mudam: confira antes de cada oferta da disciplina.



Estes quatro slides saíram da teoria para deixar as duas partes focadas em conceitos. Recomendo apresentá-los nos primeiros 10 minutos da prática: sem HTTP e async, os TODOs do app viram cópia sem entendimento, e sem CORS o primeiro erro no Chrome trava a turma.



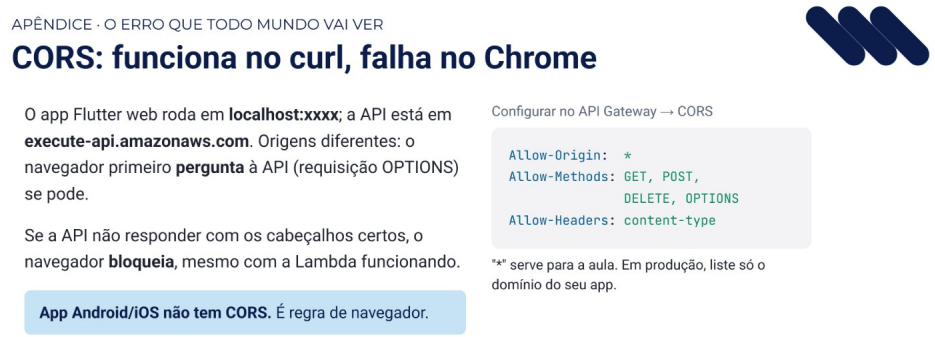
Onde fica cada coisa (4 min). Este é o slide de segurança mais importante para iniciantes. Mostre ao vivo: abra o app Flutter web, F12, aba Sources: o main.dart.js está lá para qualquer um baixar. Por isso a validação no app é só conforto; a Lambda da prática valida de novo (autor de 1 a 40 caracteres, texto de 1 a 280). A URL da API pode ficar no app porque é pública por natureza; o que a protege é autenticação e regras no backend, não segredo.



HTTP e JSON (5 min). Revisão rápida, pois muitos já viram em web. O ponto é ler um erro pelo status: 4xx = o app pediu errado; 5xx = o backend quebrou, e aí o lugar de olhar é o CloudWatch Logs da Lambda. O JSON embaixo é exatamente o formato que a API da prática devolve; peça para identificarem quais campos o app manda (autor, texto) e quais a nuvem gera (id, curtidas, criadoEm). Isso já é o design da API.



Flutter assíncrono (7 min). Esse é o conceito novo para quem está aprendendo Flutter. Future = "um valor que vai chegar depois". await = "pause esta função até chegar, mas deixe a tela viva". Sem async, a interface congelaria esperando a rede. O FutureBuilder redesenha a tela em cada um dos três estados; esquecer o estado de erro é o bug clássico (tela branca quando a rede falha). Este código é praticamente o método listar() do app da prática, que vem pronto como exemplo; eles vão escrever criar() e apagar() seguindo esse modelo.



CORS (4 min). Antecipe o erro: no console do Chrome (F12) aparece "has been blocked by CORS policy", e no Flutter aparece um ClientException genérico ("XMLHttpRequest error" / "Failed to fetch"). A Lambda da prática já devolve os cabeçalhos CORS, mas a requisição de pré-voo (OPTIONS) precisa ser atendida pelo API Gateway; por isso a configuração de CORS no API Gateway é obrigatória para quem roda no Chrome. Quem rodar no emulador Android não verá o problema, o que é uma boa demonstração de que CORS é coisa do navegador, não do servidor.